COMPILER DESIGN SEMESTER PROJECT

MiniSQL Relational Query Compiler

Detailed explanation of the assignment, the implementation, and how one SQL query travels through every compiler phase.

Course track	Project goal	Implementation
Group 8, Track H	Compile a core SQL subset into an execution plan and run it.	Python compiler pipeline with an in-memory JSON database.

Central idea. A compiler does not jump directly from source text to an answer. It transforms the input in stages. Each stage produces a representation that is easier for the next stage to understand and checks a different kind of correctness.

This project follows the assignment guideline: Scanner, Parser and AST, Semantic Analysis and Symbol Table, then Intermediate Code and Output. The final executor makes the generated plan observable by returning a result table.

What the project supports

SELECT expressions and SELECT *, FROM, table aliases, INNER and LEFT JOIN with ON, WHERE conditions, arithmetic, comparisons, AND/OR/NOT, ORDER BY, and LIMIT. The database schema and rows are loaded from JSON.

This is a compiler-design demonstration, not a complete commercial SQL database. Its value is that each compiler stage can be inspected and tested separately.

1. Overall Architecture

```
SQL source text
    |
    v
Scanner / lexer.py       characters -> tokens
    |
    v
Parser / parser.py      tokens -> AST (ast_nodes.py)
    |
    v
Semantic / semantic.py  AST -> resolved, type-checked AST + symbol table
    |
    v
Code generator           AST -> relational plan + TAC expression blocks
    |
    v
Executor                 plan + catalog -> result rows
    |
    v
Formatted output
```

The main driver is **main.py**. It loads the catalog, receives SQL from a file, command-line argument, or interactive prompt, calls the compiler, optionally prints intermediate structures, and executes the plan unless --no-run was requested.

The pipeline is coordinated by **compiler.py**. Its compile_source function calls tokenize, parse, analyze, and generate in sequence. If a phase reports errors, compilation stops before later phases that require valid input. The run function executes plans only when compilation has no errors.

Module	Responsibility
lexer.py	Recognizes keywords, identifiers, literals, operators, comments, and source positions.
parser.py	Implements recursive-descent grammar rules and panic-mode recovery at semicolons.
ast_nodes.py	Defines AST node types for expressions and SELECT query structure; can print the tree.
semantic.py	Loads schema information into a catalog; resolves names, aliases, scopes, and types.
codegen.py	Creates SCAN/JOIN/FILTER/SORT/PROJECT/LIMIT/EMIT operations and TAC blocks.
executor.py	Interprets the plan over the JSON rows and formats a result table.
errors.py	Defines phase-specific lexical, syntax, semantic, and runtime errors.

Important compiler vocabulary

• **Token:** a categorized piece of input, such as SELECT, an identifier, an integer, or an operator.

• **Grammar:** rules describing which token sequences form valid queries.

• **AST:** Abstract Syntax Tree; a structured representation of the query without irrelevant punctuation.

• **Symbol table:** information used to resolve names in the current query, especially table aliases and columns.

• **TAC:** Three-Address Code; simple instructions that store intermediate results in temporary names such as r1.

2. Scanner: Characters to Tokens

The scanner in **lexer.py** uses one master regular expression. It scans the source string, classifies each match, and emits Token objects containing a type, a value, and a line/column location.

```
SELECT e.salary * 12 -- annualize
FROM employees e;
```

The scanner recognizes SELECT and FROM as keywords; e and salary as identifiers; * as punctuation/operator; 12 as an integer; and ; as a statement terminator. The comment is discarded. Identifiers and keywords are case-insensitive in this implementation; ordinary identifiers are normalized to lowercase.

The scanner also converts numeric text to Python int/float values and turns doubled quotes inside a SQL string into a single quote. It appends an EOF token so the parser can recognize the end of input. Illegal characters and unterminated strings are reported with source positions.

Why tokenize first?

The parser should reason about language units, not individual characters. Once salary is an IDENT token, the parser does not need to rediscover where that name begins and ends.

3. Parser: Tokens to Syntax Tree

The parser in **parser.py** is a recursive-descent parser. Each method corresponds to a grammar category. For example, select_stmt parses a SELECT query, join parses a join clause, and primary parses a literal, column, or parenthesized expression.

Expression parsing is layered to enforce operator precedence. From lower to higher precedence, this grammar handles OR, AND, NOT, comparisons, addition/subtraction, multiplication/division, unary minus, and primary expressions. Thus **1 + 2 * 3** is interpreted as **1 + (2 * 3)**.

The parser constructs dataclass nodes defined in **ast_nodes.py**: Select, JoinClause, SelectItem, Column, Literal, Unary, and Binary. These nodes retain source locations and later receive semantic type and name-resolution information.

If syntax is wrong, the parser raises a ParseError. At the program level it records the error and skips tokens until a semicolon or EOF, allowing later statements to be checked. This is called panic-mode error recovery.

```
SELECT name employees;
-- Syntax error: FROM is missing
```

4. Semantic Analysis: Does the Query Make Sense?

A syntactically valid query can still contain mistakes. The semantic analyzer in **semantic.py** checks names, scope, ambiguity, and type rules using the schema loaded from **data/sample_db.json**.

Catalog and symbol table

The catalog stores table schemas and rows. For example, employees has id: INT, name: TEXT, salary: FLOAT, age: INT, and dept_id: INT. departments has id: INT, name: TEXT, and city: TEXT.

For a query that says FROM employees e JOIN departments d, the query symbol table maps e -> employees and d -> departments. A qualified reference e.salary is looked up only in employees; an unqualified reference such as name is checked across all tables in scope.

Checks the analyzer performs

• Table exists: a misspelled table name is rejected.

• Alias is unique: declaring the same alias twice is rejected.

• Column exists and is in scope: unknown aliases and unknown columns are reported.

• No ambiguous unqualified column: if both joined tables have name, use e.name or d.name.

• Types fit their operators: arithmetic needs numbers; comparisons require compatible numeric types or two TEXT values.

• Conditions are Boolean: WHERE and JOIN ON must evaluate to BOOL; AND, OR, and NOT also require Boolean operands.

• SELECT * is expanded into explicit column expressions, and ORDER BY can resolve a SELECT alias such as yearly.

```
SELECT name FROM employees e JOIN departments d ON e.dept_id = d.id;
-- Semantic error: name is ambiguous; qualify it as e.name or d.name
```

When a table is unknown, the analyzer suppresses follow-on column errors so one root problem does not create many confusing messages. Errors carry a phase name and source line/column via **errors.py**.

5. Code Generation: AST to TAC-Style Plan

The code generator in **codegen.py** lowers the semantically checked AST to an ordered list of PlanOp objects. Relational operations use temporary names t1, t2, and so on. Expression instructions use r1, r2, and so on.

Plan operation	Meaning
SCAN	Read rows from one table and qualify columns with the table alias.
JOIN	Combine left and right rows that satisfy the ON expression.
FILTER	Keep rows for which the WHERE expression is true.
SORT	Order rows by one or more evaluated keys.
PROJECT	Calculate and retain the requested SELECT expressions.
LIMIT	Keep at most the requested number of rows.
EMIT	Return the final header and rows.

Expression blocks contain TAC-like calculations such as r1 = e.salary * 12. A block also identifies its final result, so the executor knows which computed value is the join condition, filter condition, sort key, or projected column.

6. Full Query Walkthrough

Use this representative query from **examples/valid.sql**. It joins employees to departments, filters by city and age, calculates annual salary, sorts, and limits the result.

```
SELECT e.name, d.name AS department, e.salary * 12 AS yearly
FROM employees e JOIN departments d ON e.dept_id = d.id
WHERE d.city = 'Addis Ababa' AND e.age < 40
ORDER BY yearly DESC LIMIT 3;
```

Phase-by-phase reasoning

Scanner: Emits tokens for SELECT, qualified identifiers, AS, FROM, JOIN, ON, comparison operators, the string literal, AND, ORDER BY, DESC, LIMIT, and integer literals.

Parser: Builds a Select node with three output items; source employees e; one INNER JOIN to departments d; ON expression e.dept_id = d.id; WHERE expression (d.city = text) AND (e.age < 40); one descending order key; and limit 3.

Semantic analysis: Resolves e and d, checks that both join columns are INT, checks city is TEXT compared with TEXT, checks age is INT compared with INT, and confirms the two conditions combine to BOOL. It resolves ORDER BY yearly to the SELECT expression e.salary * 12. Since salary is FLOAT and 12 is INT, yearly is numeric and can be sorted.

Code generation: Emits scans, inner join, filter, sort, project, limit, and emit in that order. The actual plan printed by the program is:

```
t1 = SCAN employees AS e
t2 = SCAN departments AS d
t3 = INNER JOIN t1, t2
      [JOIN:ON] r1 = e.dept_id == d.id => r1
t4 = FILTER t3
      [FILTER:WHERE]
          r1 = d.city == 'Addis Ababa'
          r2 = e.age < 40
          r3 = r1 AND r2 => r3
t5 = SORT t4 (e.salary * 12 DESC)
t6 = PROJECT t5 (name, department, yearly)
t7 = LIMIT t6, 3
EMIT t7
```

SQL = is normalized to the internal comparison operator ==. The filter computes both comparisons, then combines them with AND. The sort expression is computed before projection, so it can use the full joined row.

Executor result

The executor reads the JSON rows. Only departments in Addis Ababa qualify; employees must also be younger than 40. Tigist (age 38, Finance) and Abebe (age 34, Engineering) match. Their salary values are multiplied by 12 and sorted descending.

```
+--------+-------------+--------+
| name   | department  | yearly |
+--------+-------------+--------+
| Tigist | Finance     | 216000 |
| Abebe  | Engineering | 144000 |
+--------+-------------+--------+
(2 rows)
```

The query requested a maximum of three rows, but only two rows satisfied the filter. LIMIT does not manufacture an extra row.

7. How to Run and Test

Run these commands from the project root: **C:\Users\Enchu\Downloads\minisql**. On this PC, use **py** because the Python launcher works even though the command python is currently intercepted by the Windows Store alias.

Run the full test suite

```
py -m unittest discover -s tests -v
```

Expected ending: Ran 25 tests ... OK. This test suite covers scanner tokens and positions, parser errors and precedence, semantic errors, joins, filters, sorting, projection, limits, and division-by-zero behavior. It was run successfully for this project.

Run the sample file with every intermediate view

```
py main.py examples/valid.sql --tokens --ast --symtab --plan
```

The flags mean: --tokens shows scanner output; --ast shows the syntax tree with inferred types; --symtab shows alias-to-table mappings; --plan shows generated relational operations and TAC. Results are executed by default.

Run one inline query or start interactive mode

```
py main.py -e "SELECT * FROM departments;"
py main.py
```

The first command executes one query. The second starts the MiniSQL prompt. At sql>, enter a SQL statement ending with a semicolon; type exit to quit.

Demonstrate errors

```
py main.py examples/errors.sql
```

This file deliberately demonstrates semantic mistakes such as an unknown table, unknown or ambiguous columns, incompatible types, a non-Boolean WHERE clause, and an unknown alias. The program prints phase-specific errors rather than a result table.

8. Error Categories and Debugging

Phase	Example issue	What to inspect
Lexical	Illegal character or unterminated string.	Token patterns and line/column tracking in lexer.py.
Syntax	Missing FROM, JOIN ON, or closing parenthesis.	Grammar method and expected token in parser.py.
Semantic	Unknown table/column, ambiguous column, wrong types.	Catalog schema and alias/type checks in semantic.py.
Runtime	Division by zero while evaluating a row.	TAC evaluation in executor.py.

A good debugging habit is to display the earliest useful compiler representation. If the token stream is wrong, inspect the scanner. If tokens look right but the AST is wrong, inspect parser precedence and grammar. If the AST is right but the query is rejected, inspect semantic resolution and types. If the plan is right but output is wrong, inspect the executor.

9. Match to the Assignment Rubric

Assignment category	Weight	Where it appears in this project	Defense evidence
Scanner / Lexer	15% | 4.5 marks	minisql/lexer.py	Run --tokens; explain keyword, identifier, number, string, and location recognition.
Parser and AST	20% | 6 marks	minisql/parser.py; ast_nodes.py	Show --ast; explain grammar, precedence, and syntax recovery.
Semantic analysis	20% | 6 marks	minisql/semantic.py	Show --symtab; demonstrate unknown, ambiguous, and type-invalid references.
Code generation	20% | 6 marks	minisql/codegen.py; executor.py	Show --plan, then show query result; connect t steps and r temporaries.
Project report	10% | 3 marks	REPORT.md and this explanation	Explain grammar, architecture, supported language, test procedure, and limits.
Group defense/demo	15% | 4.5 marks	Live CLI demonstration and team answers	Run tests, then one successful query and one intentional error.

10. Suggested Live Demonstration

A clear 3-5 minute demonstration can follow this sequence:

• **Introduce the input and scope.** Say this is a SQL subset compiler for SELECT, WHERE, and JOIN; the rows are in a JSON catalog.

• **Show the tests.** Run the unittest command and point out that scanner, parser, semantic, and execution cases are covered.

• **Show the full pipeline.** Run examples/valid.sql with --tokens --ast --symtab --plan. Pick one query and follow the stages.

• **Explain the generated plan.** SCAN reads rows, JOIN combines matching rows, FILTER applies WHERE, SORT orders them, PROJECT forms output columns, LIMIT truncates, EMIT returns the result.

• **Show error handling.** Run examples/errors.sql and explain why the errors belong to different compiler phases.

• **State a limitation honestly.** Mention unsupported SQL features and one possible future extension, such as GROUP BY or predicate pushdown.

Short defense explanation to practice

```
"Our MiniSQL program compiles a subset of SQL in stages. The lexer turns text into tokens. The recursive-descent parser
```

11. Questions You May Be Asked

Why have separate compiler phases? Each phase has one responsibility. This makes errors easier to locate and lets tokens, ASTs, symbol tables, and plans be inspected independently.

Why use an AST instead of executing directly from tokens? The AST captures the grammatical meaning of the query. Semantic analysis and code generation can work on that structure without reparsing the raw text.

What does the symbol table store? This implementation stores query aliases mapped to table names. Column types are looked up in the catalog schema.

How is operator precedence implemented? The recursive-descent parser uses separate methods for OR, AND, NOT, comparisons, additive, multiplicative, unary, and primary expressions.

What is three-address code here? Expression calculations use a destination temporary and one or two operands, such as r1 = e.salary * 12. These blocks live inside relational plan operations.

How does LEFT JOIN differ from INNER JOIN? INNER JOIN emits rows with a matching ON condition. LEFT JOIN also retains an unmatched left row and fills right-side columns with NULL.

What is not supported? The implementation does not include GROUP BY, aggregates, subqueries, INSERT, or UPDATE. Joins use a nested-loop method rather than an optimizer.

12. Limitations and One Code Detail to Review

The language intentionally covers a core subset, not all SQL. The executor uses nested loops for joins, and the project does not implement a query optimizer. NULL behavior is SQL-like in a few places but simplified; this should not be presented as full SQL NULL semantics.

Potential division type mismatch: semantic.py currently assigns INT when both operands of division are INT, while executor.py uses Python division, which produces a floating-point value. If asked about type guarantees, explain that runtime behavior and the static type rule should be aligned if integer division is part of the expected language definition. The existing division test checks division by zero, not the inferred type of successful integer division.

This is a useful improvement to identify in a defense: explain the design, acknowledge the mismatch, and propose either making integer division produce FLOAT statically or defining integer-division semantics and implementing them consistently.

13. Final Checklist

• Know the phase order: scanning -> parsing -> semantic checks -> code generation -> execution.

• Be able to explain one query at every stage, especially the JOIN example in this guide.

• Know where the schema and sample data live: data/sample_db.json.

• Run the test suite and the demo from the project root using py.

• Know the supported SQL subset, the main limitations, and one future improvement.

• Assign group members clear speaking roles for scanner, parser, semantics, code generation, and testing/documentation, as suggested by the assignment.

Source basis: Compiler Design Project assignment PDF (4 pages), project REPORT.md, and the current MiniSQL source, examples, tests, and sample database.